Fast Link-Failure Recovery in Software-Defined Networks: A Survey of Detection and Recovery Mechanisms with Two Reproduced Benchmarks

Vinaybaalaji P. N. S.

***Abstract*—Software-defined networking (SDN) separates the device that notices a link failure from the controller that decides how to route around it. The recovery schemes proposed since 2010 are, in effect, answers to the cost of that separation, judged against the 50 ms protection-switching target inherited from SONET/SDH. This survey traces those answers from the first OpenFlow measurements to programmable data planes and learning-based recovery, and organises them along three independent axes: how the failure is detected, how traffic is rerouted, and where the repair is made. We collect the recovery times that the original papers report and show that they cannot be compared directly, because they were obtained with different detection mechanisms, platforms and failure models. To put two of the canonical comparisons on a common footing, we reproduce them on one containerised Mininet/Open vSwitch testbed in 395 measured trials: restoration versus fast-failover protection (Sharma *et al.*), and BFD-driven protection with crankback and hybrid re-optimisation (van Adrichem *et al.*). Protection with 1/5/15 ms BFD recovers in a median 3.6/14.7/40.9 ms on the triangle topology, against 3.4/15.0/42.1 ms in the original. Restoration grows by about 2 ms per millisecond of one-way controller distance in both reproductions, while protection stays flat, and no scheme that relies on loss of signal recovers from a silent failure. The reproduction also exposed an implementation pitfall that offline tests and simulation did not show. We conclude that failure detection, rather than the rerouting primitive, is the binding constraint, and we outline the open problems that follow.**

***Index Terms*—Software-defined networking, OpenFlow, P4, failure recovery, fast reroute, protection, restoration, bidirectional forwarding detection, reproducibility.**

I. Introduction

SOFTWARE-DEFINED networking moves routing decisions out of individual switches and into a logically centralised controller that programs their forwarding tables [1]–[3]. It has been deployed in production, for example in Google's inter-data-centre WAN [4]. It gives operators a global view and a single point at which policy is expressed. It also changes what happens when a link fails.

Failures are routine rather than exceptional. Measurement studies of an operational IP backbone and of large data centres show that link failures are a regular part of operation [5], [6]. In a distributed IP network the router next to a failure runs the routing protocol itself, so local detection and local decision coincide. SDN breaks that coupling: the switch that observes the failure usually does not know an alternative, and the controller that knows one is at least a control-channel round trip away [7]. Each recovery scheme in this survey is a way of paying for, or avoiding, that round trip.

The yardstick has been inherited rather than derived. Carrier networks expect protection switching to complete within 50 ms, a target fixed for SDH protection architectures [8] and adopted by the first OpenFlow recovery studies as the definition of "carrier grade" [9], [10]. The terms *protection* (an alternative prepared before the failure) and *restoration* (an alternative computed after it) come from the same transport-network tradition [11].

Why another survey. The field is well covered (Table I). Surveys exist on SDN fault management in general [12], [13], on link-failure recovery [14], [15], on data-plane failures [16], on fast recovery in packet-switched networks of all kinds [17], and on rerouting with P4 switches [18]. They share one limitation: recovery times are quoted as reported by each original paper. Those numbers were obtained on different platforms, with different failure-detection mechanisms and different failure models, so placing them side by side suggests a ranking that the evidence does not support. Detection is often treated as a footnote, although in every protection scheme it dominates the recovery budget.

Contributions. This survey makes four contributions:

1) A chronological survey of SDN link-failure recovery from 2010 to 2026, organised along three independent axes: *detection*, *rerouting strategy* and *repair scope* (Sections III–V).
2) A benchmark table in which every reported recovery time is annotated with the detection mechanism, platform and failure model that produced it, and an explanation of why these numbers are not directly comparable (Section VI).
3) Two reproductions of canonical comparisons on one containerised testbed, 395 trials in total: restoration versus protection [10], and BFD-driven protection with crankback and hybrid re-optimisation [19] (Section VII). Code, measured results and a container image for R2 are public [20]; the R1 code is available on request.
4) Lessons from the reproduction, including three pitfalls that produce plausible but wrong results, and a list of open problems (Sections VII-F and VIII).

The author is a student at TIFAC-CORE in Cyber Security, School of Computing, Amrita Vishwa Vidyapeetham, Coimbatore, India (e-mail: vinaybaalaji.vb@gmail.com).

Code, measured results and a container image that reproduce R2 (Section VII) are available at https://github.com/baalajivinay/sdn-fast-recovery. The R1 code is available from the author on request.

TABLE I
EXISTING SURVEYS RELATED TO SDN FAILURE RECOVERY

Survey	Year	Scope
Chen *et al.* [13]	2015	Fault tolerance in SDN
Fonseca and Mota [12]	2017	Fault management in SDN (all planes)
Ali *et al.* [14]	2020	SDN approaches to link-failure recovery
Chiesa *et al.* [17]	2021	Fast recovery in packet-switched networks in general
Khan *et al.* [16]	2023	Systematic review of data-plane failure recovery, 70 papers from 2010–2021
Mazloum *et al.* [18]	2023	Rerouting with P4 programmable switches
Alhiyari *et al.* [15]	2025	Link-failure detection and recovery in SDN
This survey	2026	Detection and recovery, 2010–2026, OpenFlow to P4; two canonical comparisons *re-measured* on one testbed

Organisation. Section II introduces the OpenFlow mechanisms and the recovery-time model used throughout. Section III traces the evolution of the field and introduces the taxonomy. Sections IV and V survey detection and recovery mechanisms. Section VI compiles reported benchmarks. Section VII presents the reproductions. Section VIII discusses open problems, and Section IX concludes.

II. BACKGROUND

A. The SDN Data Path

An OpenFlow switch forwards packets through a pipeline of flow tables populated by the controller over the OpenFlow channel [2], [21]. A packet that matches an entry has the entry's instructions applied; a packet that matches nothing can be sent to the controller. Port-state changes are reported to the controller in `PORT_STATUS` messages. P4 generalises the model by letting the operator define the parser and the match-action pipeline itself [22], which, as Section V shows, allows detection and rerouting logic to run entirely inside the switch. Open vSwitch (OVS) is the software switch used by most emulation studies, including ours [23], typically inside the Mininet emulator [24].

B. Group Tables and the Fast-Failover Group

Group tables were introduced in OpenFlow 1.1 [25] and are retained in OpenFlow 1.3 [21], the version most deployments and studies use. A flow entry may send a packet to a group, which holds a list of *action buckets*. For the *fast-failover* type (`OFPGT_FF`), the switch executes the first bucket whose watched port or group is live, so it "change[s] forwarding without requiring a round trip to the controller" [25]. If no bucket is live, packets are dropped. A port is live when its `OFPPS_LIVE` flag is set. The specification allows port liveness to be "managed by code outside of the OpenFlow portion of a switch", such as a keep-alive mechanism, and requires at minimum that a port whose link is down is not live [21]. In practice liveness is the carrier state unless a detector such as BFD is attached.

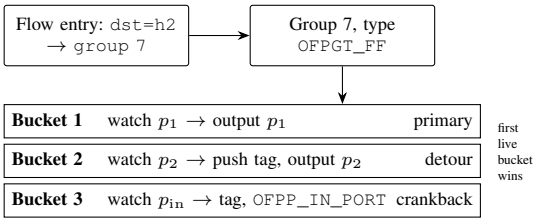


Fig. 1. An OpenFlow fast-failover group. The switch executes the first bucket whose watched port is live. Bucket 2 is a local detour; bucket 3 returns the packet upstream (crankback) and must use the reserved port `OFPP_IN_PORT` [21].

Fig. 1 shows the structure. The fast-failover group is the primitive on which nearly all OpenFlow-era protection rests. One detail matters for local repair: a packet is not forwarded back out of the port it arrived on unless the action uses the reserved port `OFPP_IN_PORT`, which "must be explicitly used in order to send back out of the input port" [21]. Section VII shows the consequence when this is overlooked.

C. Failure Model

We consider link failures in the data plane. Following the distinction used by van Adrichem *et al.* [19], a failure is either *carrier-visible*, when the interface loses signal and the switch can observe it locally, or *silent*, when frames are lost while both ends still report the link as up (for example, a fault behind a transceiver or a misconfiguration). A failure of the control channel itself is a separate case, discussed in Section IV-F.

D. Anatomy of Recovery Time

Recovery time is the interval during which traffic of an affected flow is lost. For restoration it decomposes as

$$T_{\text{rest}} = T_{\text{det}} + T_{\text{notify}} + T_{\text{comp}} + T_{\text{inst}}, \tag{1}$$

where T_{notify} and T_{inst} each include one controller-to-switch propagation delay d, so that $T_{\text{rest}} \geq T_{\text{det}} + 2d$. For protection the controller is absent:

$$T_{\text{prot}} = T_{\text{det}} + T_{\text{sw}}, \tag{2}$$

where T_{sw} is the switch's bucket change-over time. Fig. 2 contrasts the two. Equations (1) and (2) explain two results that recur throughout this survey. Restoration depends on controller placement [7], and protection depends almost entirely on detection. Other metrics matter as well: packet loss, the forwarding state consumed in ternary content-addressable memory (TCAM), path stretch, and congestion on the post-recovery path.

III. EVOLUTION OF THE FIELD AND A TAXONOMY

A. Four Phases, 2010–2026

Fig. 3 places the works surveyed here on a time line. Four phases can be distinguished. Each was a response to the limit the previous one reached.

Phase 1: measuring the cost of centralisation (2010–2014). The first studies asked whether OpenFlow could meet carrier requirements at all. Desai and Nandagopal showed that

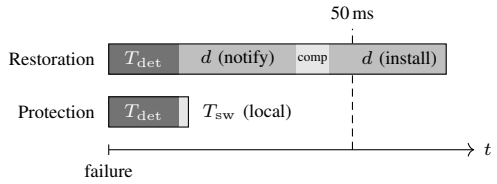


Fig. 2. Components of recovery time. Restoration pays the detection time plus at least one control-channel round trip ($2d$); protection pays the detection time plus a local change-over. The drawing is illustrative, not to scale.

a link failure in a centralised architecture can cut switches off from the controller or create routing loops, and proposed informing only the relevant switches so that they stop sending traffic toward the failed link [26]. The Ghent group measured controller-driven restoration and concluded that "it will be hard to achieve 50 ms restoration in large networks serving many flows", so protection is required [9], [27]. Their journal paper made the canonical comparison of the two strategies [10]. Kempf *et al.* moved link monitoring into the switch so that the controller is not loaded by millisecond-rate probes [28], and van Adrichem *et al.* showed that per-link BFD with pre-installed fast-failover groups cuts recovery by an order of magnitude [19].

Phase 2: making protection affordable (2013–2018). With protection established as the fast option, the cost of its pre-installed state became the problem. Work in this phase reduced backup state through local rerouting and rule sharing [29], [30], table compression [31], source routing [32], topology co-design [33] and flow aggregation [34]. Other work gave provable guarantees under multiple failures [35]–[37]. Stateful extensions let switches run their own detection logic [38], [39].

Phase 3: programmable data planes (2016–2022). P4 [22] removed the fixed pipeline. Detection could then be inferred from traffic [40], fast-reroute primitives could be implemented directly in hardware pipelines [41], [42], and loss during failover could be eliminated altogether [43], [44]. Route computation also moved into the data plane [45], [46].

Phase 4: quality of the recovered network (2017–2026). Recent work asks what the network looks like *after* recovery. It covers congestion on backup paths [47], [48], load distribution under multiple failures [49], and learning-based prediction and self-healing [50], [51]. The phases overlap in time: congestion-aware recovery began in 2017, while P4 work continues.

B. Three Axes of Classification

Most surveys classify schemes by a single axis, usually protection versus restoration [14], [15]. Mazloum *et al.* instead split rerouting into a *feedback* phase, in which the problem is noticed, and a *rerouting* phase, in which forwarding reacts [18]. We adopt that separation and add the scope of repair, which gives three axes that vary independently:

1) **Detection**: how, where and how quickly the failure becomes known. Options include controller-driven discovery, loss of signal (LoS), BFD, in-switch probing and traffic inference (Section IV).

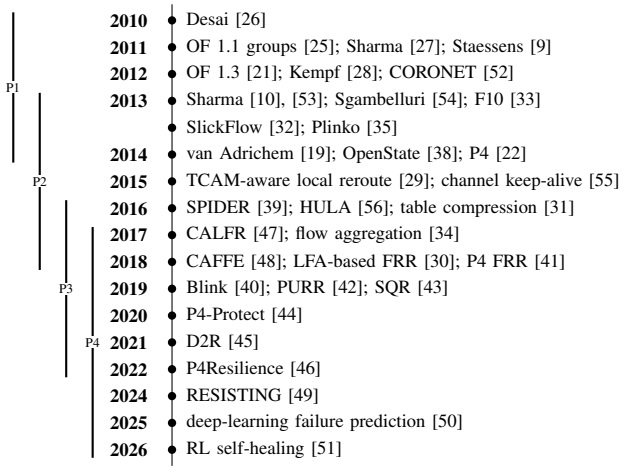


Fig. 3. Time line of SDN link-failure recovery, 2010–2026, by year of publication. Bars on the left mark the four phases of Section III-A: P1 cost of centralisation, P2 affordable protection, P3 programmable data planes, P4 quality of the recovered network.

2) **Rerouting strategy**: when the alternative is computed. Restoration computes it after the failure, protection before, and hybrids do both (Section V).
3) **Repair scope**: where the repair is applied. Path protection repairs at the ingress, segment protection at a segment end-point, and link or local protection at the node adjacent to the failure (Section V-C).

The axes matter for interpreting results. Two schemes that differ only in rerouting strategy can be compared directly; a scheme with fast detection and slow rerouting cannot be compared with one that has the opposite combination by quoting a single number. Section VII exploits this by varying one axis at a time.

IV. Failure Detection

Recovery cannot complete before detection. In protection schemes detection is most of the recovery time (Eq. (2)). This section surveys detection mechanisms in order of where they run, from the controller down to the data plane. Table II summarises them.

A. Controller-Driven Discovery

The default topology service of most controllers discovers links by sending LLDP frames through the switches and observing where they arrive. Kempf *et al.* point out that this model has "serious scalability limitations": fast recovery would need monitoring messages at millisecond intervals on every link, all processed by the controller [28]. Controller-driven discovery is therefore unsuited to a 50 ms budget. A practical consequence for experimenters is that a restoration controller driven by periodic discovery measures the discovery timer, not the recovery mechanism.

B. Loss of Signal

Loss of signal is passive and free. The interface reports the carrier drop and the switch updates the port's liveness,

which a fast-failover bucket can watch directly. LoS has two limitations. First, it sees only the local link, and only failures that remove the carrier. Second, its speed depends on the hardware and driver. Van Adrichem *et al.* measured average LoS detection of 4759 ms on Open vSwitch and 1697 ms on a Pica8 switch with physical interfaces [19], while an overview by the same group reports 100–200 ms for the setup of Sharma *et al.* [57]. In emulation over virtual Ethernet pairs, carrier loss is propagated almost instantly. This difference is a major source of optimistic results (Section VII-G).

C. Bidirectional Forwarding Detection

BFD runs a lightweight hello protocol between two systems [58]. In asynchronous mode each side sends control packets at a negotiated interval T; a session is declared down when no packet has arrived for the detection time $M \cdot T$, where M is the detection multiplier. Transmissions are jittered by reducing each interval by 0–25% [58]. Because the failure can occur at any point between two received packets, the time from failure to detection lies roughly between $(M-1)T$ and MT. With the common $M = 3$, a 50 ms budget for detection alone needs $T \leq 16.7$ ms.

Two placements are used in SDN. *Per-path* sessions between the ingress and egress of each protected path monitor the whole path, so the number of sessions grows with the number of protected paths; Sharma *et al.* used this placement and reported 40–44 ms detection [10], [57]. *Per-link* sessions between neighbours scale with the number of links instead and allow much shorter intervals; van Adrichem *et al.* used intervals of 1, 5 and 15 ms [19]. In OVS, a BFD session that goes down clears the port's liveness, so the fast-failover group reacts to it exactly as it would to a carrier loss. BFD's cost is overhead: packets on every monitored link, and processing in the switch. In our reproduction, BFD run by the OVS user-space daemon at $T = 1$ ms gave anomalous results in 4 of 20 trials, including one failure that was never detected (Section VII).

D. In-Switch and Stateful Monitoring

Several designs move probe generation into the switch without relying on a standard protocol. Kempf *et al.* extended OpenFlow 1.1 with a switch-side monitoring function and reported data-plane recovery "within 50 milliseconds" [28]. Lee *et al.* combined pre-planned paths with software-based failure detection and localisation [59]. SPIDER uses the stateful OpenState pipeline [38] to send periodic heartbeats, detect remote failures and reroute in the switch, even when the controller is unreachable. Its journal version reports detection and recovery delays of "few milliseconds or less", with a configurable trade-off between probe overhead and responsiveness [39], [60].

E. Inference from Traffic

Blink takes a different approach. It sends no probes and instead infers failures from the behaviour of the affected TCP flows. A disrupted flow retransmits the same segment at exponentially spaced intervals, and when many flows do this at once the result is a strong failure signal. Blink detects this pattern in the data plane of a P4 switch and reroutes, achieving "sub-second rerouting for large fractions of Internet traffic" [40]. Inference needs no cooperation from neighbours and can detect remote failures. It is, however, much slower than link-level BFD, and it depends on traffic: a link that carries no TCP traffic gives no signal.

F. Control-Channel Failures

When the failed link also carries control traffic (in-band control), the switch can lose its controller. Sharma *et al.* showed that restoration of in-band networks does not recover within 50 ms and that restoring the control path delays restoration of the data path, whereas protecting both control and data traffic meets the carrier-grade requirement [53]. The same group notes that the OpenFlow echo timeout used to detect a lost channel is at least 1 s in their setting [57]. Kotani and Okabe proposed adaptive keep-alives that detect channel failures quickly without loading the controller [55].

G. Why Detection Must Be Normalised

Table II spans more than three orders of magnitude, from milliseconds to seconds. Two papers that report recovery times for the same rerouting strategy with different detection mechanisms are therefore measuring mostly their detectors. LoS on emulated links is nearly instantaneous and makes a protection scheme look several times faster than the same scheme with BFD (Table V); LoS on physical NICs, at seconds [19], would make it look slower than BFD-driven restoration. This is why Section VI annotates every reported number with its detection mechanism, and why our reproductions hold detection fixed while varying one other parameter.

V. Recovery Mechanisms

This section follows the rerouting-strategy axis and, within it, the repair-scope axis. Table III at the end of the section gives one row per scheme.

A. Restoration

In restoration the switch reports the failure, the controller computes new paths and installs them (Eq. (1)). Its advantages are that no backup state occupies the switch until it is needed, and that the new path can reflect the current topology and load. Its disadvantage is that recovery time grows with the control-channel delay and with the number of rules to rewrite. Staessens *et al.* found that OpenFlow "can restore traffic quite fast" in an emulated carrier-grade network but that 50 ms will be hard to reach in large networks with many flows [9]. Sharma *et al.* proposed a faster restoration mechanism [27], and their later comparison showed restoration times that grow with the number of affected flows and can exceed 1000 ms [10], [57]. Later restoration work reduces the number of flow operations, for example by aggregating all disrupted flows into one rerouted "big" flow [34] and choosing the local reroute path to avoid congestion [47].

TABLE II
FAILURE DETECTION MECHANISMS USED IN SDN RECOVERY

Mechanism	Runs in	Silent failures	Remote failures	Reported or derived detection time	Main cost
LLDP discovery	controller	yes	yes	discovery period; too slow at scale [28]	controller processes every probe [28]
Loss of signal	PHY / driver	no	no	4759 ms (OVS), 1697 ms (Pica8) [19]; 100–200 ms [57]; $\approx$0 on veth (ours)	none
BFD, per path	ingress/egress	yes	yes	40–44 ms [10], [57]	sessions grow with protected paths
BFD, per link	adjacent switches	yes	no	$\approx (M-1)T$ to MT; T = 1–15 ms [19], [58]	probes on every link; switch CPU
In-switch heartbeats	stateful pipeline	yes	yes	"few ms or less" [60]	stateful switch support
OAM extension	switch	yes	per design	within a 50 ms recovery [28]	protocol extension
TCP-signal inference	P4 data plane	yes	yes	sub-second [40]	needs TCP traffic; spoofable
Channel keep-alive	switch, controller	—	control channel	adaptive [55]; echo timeout $\geq$1 s [57]	keep-alive traffic

B. Protection

In protection the alternative is installed in advance, usually as a fast-failover group (Fig. 1), and the switch changes over on its own (Eq. (2)). Sharma *et al.* implemented 1:1 path protection with OpenFlow group tables and per-path BFD and showed that it meets the 50 ms requirement "even in a large-scale network serving many flows" [10], [53]. Protection removes the controller from the critical path, so recovery becomes insensitive to controller placement, as our measurements confirm (Section VII). The price is state: every protected flow consumes extra entries in every switch on its backup path. Mohan *et al.* note that TCAM "can hold only a few thousands of rules at a switch since it is expensive and power hungry" [29].

C. Repair Scope and the Crankback Problem

Path protection repairs at the ingress. It needs one backup per flow and a detector that reports failures to the ingress (per-path BFD in [10]). *Segment protection* provides a backup for each segment of the working path. Sgambelluri *et al.* implemented it with working and backup entries at different priorities and an auto-reject mechanism that removes failed entries. Their recovery time is "only due to the failure detection time, i.e., a few tens of milliseconds" [54]. *Link* or *local* protection repairs at the node adjacent to the failure, so a link-local detector such as per-link BFD suffices.

Local repair has a structural difficulty. The node adjacent to the failure may have no alternative except the way the packet came. Van Adrichem *et al.* handle this with *crankback*: the packet is sent back upstream to a node that has an alternative, and the detour path is followed from there [19]. Without marking, such packets can loop, and the usual remedy is to tag them (for instance with a VLAN or MPLS label) so that downstream nodes recognise a packet that is already on a detour. Crankback also meets the ingress-port rule of Section II-B: the crankback bucket must output to `OFPP_IN_PORT`. Mohan *et al.* compute forward and backward local reroutes that share rules between primary and backup paths and report up to 75% fewer additional rules per protected flow in simulation [29]. Merling *et al.* carry loop-free alternates [61] and remote LFAs [62] from IP fast reroute [63] over to OpenFlow. They find that classic remote LFAs create many loops under node failures, and that one of their explicit-path variants protects against all single link and node failures with few additional entries per node [30].

D. Reducing the State of Protection

A large part of the literature can be read as "protection, but cheaper". *Source routing* keeps state only at the edge. SlickFlow encodes a primary and an alternative path as segments in packet header fields, so a switch on the primary path can redirect a packet without the controller [32]. *Table compression* attacks the cost directly. Stephens *et al.* report compression ratios of 2.10$\times$ to 19.29$\times$ for multi-failure fast-failover tables [31]. *Topology co-design* avoids enumerating backups: F10 combines a FatTree-like topology, failover protocols and failure detection, and reports less than one seventh of the packet loss of existing schemes after link and switch failures [33]. CORONET recovers from multiple link failures in the data plane; its prototype was built on NOX and evaluated in Mininet [52]. *Aggregation* lets many flows share one backup rule [34], [47].

E. Resilience to Multiple Failures

Single-failure protection is the common case. Several works aim for guarantees beyond it. Plinko builds forwarding tables that are "provably resilient against t link failures"; with tables comparable to top-of-rack switches it scales to networks of up to ten thousand hosts [35]. Borokhovich *et al.* study which connectivity guarantees local fast-failover rules can provide and give algorithms for them [36]. Chiesa *et al.* study how much resilience static forwarding tables can provide [37]. In the P4 era, P4Resilience targets recovery from multiple failures in the programmable data plane [46], and RESISTING combines ECMP with fast reroute and distributes flows evenly with no packet loss under up to three simulated link failures [49].

F. Stateful and Programmable Data Planes

OpenFlow switches can be configured but not programmed, so the schemes above encode their logic in a fixed pipeline. Stateful data planes relax this. OpenState lets the switch keep

per-flow state and change its own behaviour [38], which SPIDER uses for in-switch detection and rerouting that does not depend on the controller [39], [60]. P4 [22] goes further:

- Sedar *et al.* present a fast-reroute primitive for P4 that is efficient in processing latency, memory and throughput [41]. PURR extends this to reconfigurable switches [42].
- Blink detects and reroutes around remote failures entirely in the data plane and runs on a Tofino switch [40].
- SQR performs packet-loss recovery inside the switch: it diverts affected flows to alternative paths and "completely eliminates packet loss during link failures", reducing tail flow-completion time by up to four orders of magnitude on a Tofino testbed [43].
- P4-Protect brings 1+1 protection to IP networks: traffic is duplicated over two disjoint paths and the receiver forwards one copy, which "prevents almost any packet loss in case of failures". It is implemented for BMv2 and Tofino [44].
- D2R performs policy-compliant route computation in the data plane, so the network does not wait for the control plane after a failure [45].

Mazloum *et al.* survey this branch in depth [18].

G. Hybrid and Congestion-Aware Recovery

Hybrid schemes combine an immediate local change-over with later controller action. Van Adrichem *et al.* let the controller recompute optimal paths once traffic is safe on the detour [19]. A pre-computed backup, however, is chosen without knowledge of the load at failure time. CALFR reassigns disrupted flows to local reroute paths chosen to avoid congestion, formulated as an integer linear program with an efficient heuristic [47]. CAFFE pre-installs protected paths planned to minimise the maximum link load after any protected failure, so switches can detour from neighbouring nodes without waiting for the controller [48]. HULA shows that utilisation-aware path choice can be made in a programmable data plane [56].

H. Learning-Based Recovery

The most recent line of work applies machine learning. Bishla and Kumar predict link failures in hybrid SDN with a stacked auto-encoder and gated recurrent units and report 96% accuracy [50]. Makiyah *et al.* train a reinforcement-learning agent for self-healing in an emulated SDN; it selects fault-appropriate actions in over 85% of fault-state time steps [51]. These results are measured as prediction accuracy or action quality, not as recovery time. Any model placed inside the outage window adds its inference latency to Eq. (1) or (2), so its role is most plausible before a failure (pre-positioning backup state) or after one (re-optimisation), rather than on the critical path. We return to this in Section VIII.

VI. Reported Benchmarks and Their Comparability

Table III is the kind of table that surveys usually present, and it invites the reader to rank schemes by the last column. The ranking would be misleading for four reasons.

Detection differs. Sgambelluri *et al.* state that their recovery time is "only due to the failure detection time" [54], and the 15 ms of van Adrichem *et al.* at $T = 5$ ms lies inside the BFD detection window [19]. In both cases the number characterises the detector more than the rerouting scheme. Section IV-G showed that detection alone spans three orders of magnitude.

Platforms differ. Software switches in emulation, OVS on physical servers, and hardware P4 targets have different change-over times and different LoS behaviour. LoS on physical NICs took seconds in [19] but is nearly instantaneous on virtual links.

Metrics differ. Recovery time, packet loss, compression ratio, rule count, flow-completion time and prediction accuracy all appear in the last column. They cannot be converted into one another.

Controller distance is rarely reported. For any scheme in which the controller is on the critical path, recovery time grows with the control-channel delay (Eq. (1)). An emulated controller on the loopback interface has a round-trip time of microseconds, which makes restoration look almost as fast as protection.

These points do not invalidate any individual result. They mean that comparisons *across* papers should be read qualitatively, and that quantitative comparison requires re-measurement under a common setup. The next section does this for the two most-cited comparisons in the field.

VII. Reproduced Benchmarks on a Common Testbed

A. Testbed and Method

Both reproductions use the same stack, packaged in a Docker container: Mininet 2.3.0 [24], Open vSwitch 2.17.5 [23] with the kernel datapath, and the os-ken 4.2.1 controller framework (the maintained fork of Ryu). The controllers are given the topology statically rather than discovering it with LLDP, so that discovery timers cannot contaminate restoration results. This favours restoration, the slower scheme.

Controller distance. The OpenFlow channel runs over the loopback interface, where the round-trip time is microseconds. We therefore emulate a one-way controller distance d with `netem` on the loopback interface. Host traffic crosses virtual Ethernet pairs between network namespaces and is not delayed. Every result below states d.

Measurement. A constant-rate UDP stream with sequence numbers runs between the hosts of the affected flow: 1000 packets/s (1 ms resolution) in R1 and 5000 packets/s (0.2 ms resolution) in R2. Recovery time is the number of packets missing in the largest gap that ends after the failure, multiplied by the send interval. This is the loss-based measure used by van Adrichem *et al.* [19]. A gap-based estimate from arrival times serves as a cross-check.

Failures. A *carrier* failure takes the link down, which on virtual Ethernet is visible to both ends immediately. A *silent* failure drops all frames in both directions with `tc` ingress filters while the carrier stays up, so that only an active detector can see it. This matches what the recovering

TABLE III
REPRESENTATIVE SDN RECOVERY SCHEMES. DETECTION: LOS = LOSS OF SIGNAL, BFD-P/BFD-L = PER-PATH/PER-LINK BFD, HB = IN-SWITCH HEARTBEAT, INF = TRAFFIC INFERENCE. SCOPE: P = PATH, S = SEGMENT, L = LINK/LOCAL. "CTRL" MARKS WHETHER THE CONTROLLER IS ON THE RECOVERY CRITICAL PATH. — = NOT STATED IN THE SOURCES CONSULTED, OR NOT APPLICABLE.

Scheme	Year	Detection	Strategy	Scope	Ctrl	Platform	Reported result
Desai [26]	2010	—	failure notification	—	partly	OpenFlow switches	relevant switches informed sooner than via controller
Staessens [9]	2011	—	restoration	P	yes	emulation	50 ms hard in large networks with many flows
Kempf [28]	2012	OAM	protection	P	no	OF 1.1 extension	recovery within 50 ms
CORONET [52]	2012	—	multi-failure recovery	—	—	NOX, Mininet	prototype; multiple link failures
Sharma [10]	2013	LoS / BFD-P	restoration vs. 1:1 protection	P	yes / no	emulation	protection within 50 ms; restoration can exceed 1000 ms [57]
Sharma (in-band) [53]	2013	LoS / BFD-P	both, control+data	P	yes / no	emulation	only protection meets 50 ms
Sgambelluri [54]	2013	—	protection	S	no	emulation, testbed	a few tens of ms (= detection time)
SlickFlow [32]	2013	—	source-routed backup	P	no	OVS, Mininet	data-centre topologies
F10 [33]	2013	co-designed	cascaded failover protocols	L	no	incl. trace-driven	$<1/7$ packet loss of prior schemes
Plinko [35]	2013	—	t-resilient tables	L	no	—	up to $\sim 10^4$ hosts with ToR-class tables
van Adrichem [19]	2014	BFD-L	protection + crankback, hybrid	L	no	24 OVS servers, 1 Gb/s	3.4/15.0/42.1 ms at T=1/5/15 ms; 13.6 ms on USNET
Mohan [29]	2015	—	TCAM-aware local reroute	L	no	simulation	up to 75% fewer extra rules
Stephens [31]	2016	port state	compressed multi-failure FF	L	no	datacentre tables	2.10–19.29× compression
SPIDER [39], [60]	2016	HB	stateful protection	L	no	OpenState, P4	few ms or less
CALFR [47]	2017	—	congestion-aware local reroute	L	yes	emulation	fast recovery without congestion
CAFFE [48]	2018	—	congestion-aware protection	L	no	experiments	lower post-recovery link load
Merling [30]	2018	port state	LFA-based FRR	L	no	topology study	all single link/node failures
Blink [40]	2019	Inf	data-plane reroute	—	no	P4, Tofino	sub-second rerouting
SQR [43]	2019	—	in-network loss recovery	L	no	P4, Tofino	no loss; tail FCT up to $10^4\times$ lower
P4-Protect [44]	2020	—	1+1 protection	P	no	BMv2, Tofino	almost no packet loss
D2R [45]	2021	—	in-switch route computation	—	no	P4	policy-compliant under failures
P4Resilience [46]	2022	—	multi-failure recovery	—	no	P4	multiple failures
RESISTING [49]	2024	—	ECMP + FRR	L	no	P4 (simulated failures)	no loss with up to 3 failures

switch experienced in [19], where LoS on physical NICs took seconds.

B. R1: Restoration versus Protection

R1 reproduces the central comparison of Sharma *et al.* [10]: controller-driven restoration against fast-failover protection, with detection held fixed. The topology is a six-switch ring. The ingress link of the primary path is cut, and both schemes detect the failure by LoS. In ten trials per scheme at $d = 25$ ms, restoration recovered in 55.70 ± 1.42 ms (median 55.0 ms) and met the 50 ms target in 0 of 10 trials. Protection recovered in 3.00 ± 0.47 ms (median 3.0 ms) and met it in 10 of 10. An independent repeat with fresh containers gave 56.70 ± 2.63 ms and 3.10 ± 0.57 ms. The ratio of roughly $18\times$ and the fact that the two schemes fall on opposite sides of 50 ms reproduce the qualitative finding of [10]. A sweep over d is shown in Fig. 5 and discussed in Section VII-D.

C. R2: BFD-Driven Protection, Crankback and Hybrid

R2 reproduces van Adrichem *et al.* [19], which R1 cannot address: active detection, silent failures, failures away from the ingress, and larger topologies. BFD runs on every link with interval $T \in \{1, 5, 15\}$ ms and the OVS multiplier $M = 3$. Fast-failover groups hold the primary and backup next hops. When a node has no local alternative, the backup bucket cranks traffic back upstream with a VLAN tag that identifies the failed link, and every switch on the detour has a tag-specific rule. The original paper reports no restoration baseline, so we add one with the same detector, as well as a hybrid scheme in which the controller re-optimises after the local change-over. Four experiments were run, 325 trials in total:

- **E1**, triangle (Fig. 3a of [19]), 10 trials per configuration: LoS versus BFD, carrier versus silent failures, restoration versus protection.
- **E2**, five-node ring (Fig. 3b of [19]), interior link C–D cut, 10 trials: crankback, restoration and hybrid. Links have 1 ms delay.
- **E3**, USNET with 24 nodes and 43 links, 30 trials, each

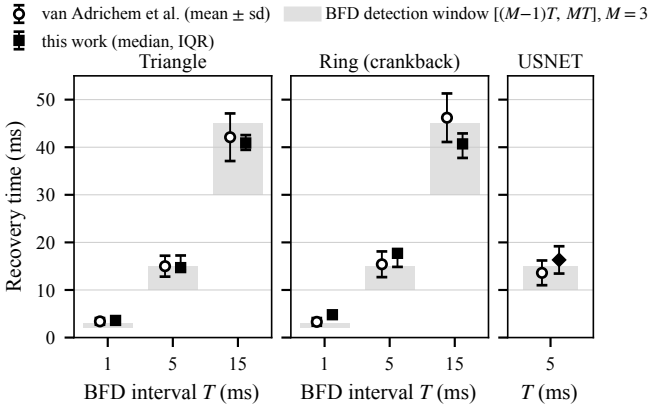


Fig. 4. Protection with per-link BFD under a silent failure: van Adrichem *et al.* [19] (open circles, mean ± sd) against this reproduction (filled squares, median and inter-quartile range; USNET: filled diamond, mean ± sd over 30 trials). Shaded bands mark the BFD detection window for $M = 3$.

TABLE IV
R2: RECOVERY TIME (MS) OF PROTECTION WITH PER-LINK BFD UNDER A SILENT FAILURE, $d = 10$ MS

Topology	$T = 1$ ms	$T = 5$ ms	$T = 15$ ms
Triangle, paper [19]	3.4 ± 0.7	15.0 ± 2.2	42.1 ± 5.0
Triangle, ours (median)	3.6[a]	14.7	40.9
Triangle, ours (mean ± sd)	—	15.3 ± 2.5	41.1 ± 4.1
Ring, paper [19]	3.3 ± 0.8	15.4 ± 2.7	46.2 ± 5.1
Ring, ours (median)	4.8[b]	17.7	40.7
Ring, ours (mean ± sd)	—	16.6 ± 2.9	40.6 ± 3.2
USNET, paper [19]	—	13.6 ± 2.6	—
USNET, ours (30 trials)	—	16.3 ± 2.9	41.3 ± 4.9

[a] Nine of ten trials recovered; one never did, and one lost no packets because its BFD session reported the failure only after 3 s. [b] Eight trials between 3.8 and 5.0 ms; two outliers of 70 and 941 ms.

cutting a random link on the primary path of one of the 20 node pairs that are farthest apart.

- **E4**, five-node ring, $d \in \{0, 5, 10, 25, 50\}$ ms, 5 trials per point.

Experiments E1–E3 use $d = 10$ ms.

BFD-driven protection reproduces the original. Fig. 4 and Table IV compare the results. Four of our seven central values lie inside the original's one-standard-deviation band: the triangle at all three intervals (3.6, 14.7 and 40.9 ms against 3.4 ± 0.7, 15.0 ± 2.2 and 42.1 ± 5.0 ms) and the ring at 5 ms (17.7 against 15.4 ± 2.7 ms). The other three lie just outside it: the ring at 15 ms (40.7 ms, 0.4 ms below the band), USNET (mean 16.3 ms, 0.1 ms above 13.6 ± 2.6 ms) and the ring at 1 ms (4.8 against 3.3 ± 0.8 ms). Every median lies inside the theoretical detection window $[(M-1)T, MT]$ or at most 2.7 ms above it, consistent with a small change-over overhead in OVS. BFD run by the OVS user-space daemon at $T = 1$ ms was, however, unreliable: 4 of the 20 triangle and ring trials gave anomalous results (Table IV, notes a and b). Van Adrichem *et al.* also used 5 ms on USNET [19], and we did the same.

Detection, not rerouting, decides whether recovery happens. Table V shows the triangle under both failure types. With a carrier failure, LoS-driven protection is the fastest configuration measured (2.1 ± 0.3 ms). With a silent failure, neither LoS-driven scheme recovered in any of 10 trials, because nothing reported the failure. Every BFD-driven configuration recovered.

TABLE V
R2/E1: TRIANGLE, ALL SCHEMES AND FAILURE TYPES, $d = 10$ MS, 10 TRIALS EACH (MEAN ± SD, MS)

Scheme	**Detection**	**Carrier failure**	**Silent failure**
Restoration	LoS	23.8 ± 1.9	never (0/10)
Protection	LoS	2.1 ± 0.3	never (0/10)
Restoration	BFD 5 ms	—	34.7 ± 1.1
Protection	BFD 5 ms	—	15.3 ± 2.5

Crankback works, but only if the ingress-port rule is respected. On the ring, the switch adjacent to the cut must return traffic upstream. Protection then recovers as fast as on the triangle (Table IV), so crankback costs no extra recovery time; it adds path stretch until the controller intervenes. In the hybrid scheme the controller moved traffic to the shortest remaining path 47.9 ± 1.4 ms after the cut on the ring, with recovery itself unchanged (15.7 ± 2.5 ms). On USNET, re-optimisation took 178.5 ± 15.6 ms (median 171.7 ms) and recovery 16.1 ± 2.5 ms. Restoration with the same BFD detector took 38.0 ± 2.1 ms on the ring and 42.3 ± 2.9 ms on USNET.

D. Controller Distance

Fig. 5 combines the controller-distance sweeps of both reproductions. A least-squares fit of mean restoration time against one-way distance d gives

$$\text{R1:}\quad T_{\text{rest}} \approx 1.99\,d + 5.8\,ms \qquad (R^2 = 0.999), \tag{3}$$

$$\text{R2:}\quad T_{\text{rest}} \approx 1.97\,d + 18.8\,ms \qquad (R^2 = 0.998), \tag{4}$$

for R1 (LoS, six-switch ring) and R2 (BFD with $T = 5$ ms, five-node ring). The slopes match the $2d$ term of Eq. (1), one control-channel round trip, in two independently built testbeds. The intercepts differ by 13 ms, which is consistent with the BFD detection window at $T = 5$ ms (10–15 ms) replacing near-instant LoS. Restoration crosses 50 ms at $d \approx 22$ ms with LoS and at $d \approx 16$ ms with BFD. Protection is flat at 2.2–2.8 ms (R1, LoS) and 13.6–16.8 ms (R2, BFD 5 ms) across a 50-fold change in d, and hybrid recovery stays at 14.0–17.1 ms. The two testbeds also agree at a common point: LoS restoration at $d = 10$ ms took 24.4 ± 0.6 ms on R1's six-switch ring and 23.8 ± 1.9 ms on R2's triangle.

E. The Cost of Protection

Protection's speed is paid for in forwarding state (Table VI). On USNET, protection with crankback installs 3293 flow and group entries against 576 for restoration, a factor of 5.7. The busiest switch holds 232 entries against 24. Hybrid holds the same state as protection, because it keeps the fast-failover groups and changes only the path used after re-optimisation. On hardware these entries would occupy TCAM, which is the motivation for the state-reduction techniques of Section V-D.

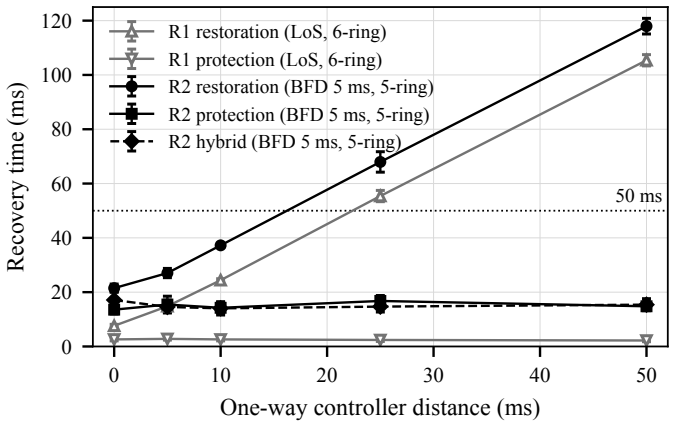


Fig. 5. Recovery time against one-way controller distance d (mean $\pm$ sd; R1: 5 trials per point, carrier failure; R2: 5 trials per point, silent failure). Restoration grows by about $2d$; protection and hybrid recovery do not depend on d.

TABLE VI
R2: FORWARDING STATE INSTALLED (FLOW ENTRIES + GROUP ENTRIES)

Topology	Restoration	Protection	Ratio	Max/switch
Triangle	6	20 (14+6)	3.3×	2 / 8
Five-node ring	10	50 (36+14)	5.0×	2 / 10
USNET	576	3293 (2716+577)	5.7×	24 / 232

Restoration installs flow entries only. Protection: flow + group entries. Max/switch: busiest switch, restoration / protection.

F. Lessons from the Reproduction

Three problems arose during the reproduction. Each produced results that looked plausible and were wrong. We report them because each is easy to repeat.

The controller was infinitely close. With Mininet's default loopback control channel, restoration in R1 completed about 1.3 ms after detection, and both schemes measured close to 0 ms. That is a correct measurement of an unrealistic network. Without the emulated controller distance, the comparison would have shown nothing.

The ingress-port rule. In the first full run of R2, protection on the ring never recovered, although BFD detected the cut. The crankback bucket output the packet to the physical port it had arrived on, and OpenFlow drops such packets unless the action uses `OFPP_IN_PORT` [21]. Our offline packet-walk tests and a simulator of the same design both reported crankback as working, because neither modelled the rule. After the fix (an ingress-port-specific rule whose crankback bucket outputs to `OFPP_IN_PORT`), every experiment was re-run, and all R2 numbers above are from the corrected code.

Anchoring the outage. An early analysis searched for the gap whose arrival times straddle the cut timestamp. Because the link goes down a fraction of a millisecond after the timestamp, that pair is usually intact, and the analysis reported 0 ms for traces with visible holes of more than 50 packets. Anchoring on the largest gap that *ends* after the failure fixed it.

G. Threats to Validity

Software switching. Absolute times are from emulation on one host. OVS runs BFD in its user-space daemon, which made $T = 1$ ms unreliable. Hardware switches run BFD in dedicated logic. Our claims are therefore relative: the slopes in Fig. 5, which side of 50 ms each scheme falls on, and agreement with the original within the stated error.

Optimistic LoS. Carrier loss on virtual Ethernet is reported instantly, whereas physical NICs took seconds in [19]. Our LoS protection results (2.1–3.0 ms) are best cases and should not be compared with hardware.

Scope. We test single link failures on otherwise idle networks. Multiple and correlated failures, node failures, control-channel failures and congestion on backup paths are not measured. R1 cuts only the ingress link. R2 maps the original's USNET flows between the east and west of the network to the 20 node pairs that are farthest apart, because node names were not available in the edge list we used.

Static topology. Neither controller uses LLDP discovery. This removes discovery delay from restoration and so biases the comparison in restoration's favour.

VIII. OPEN PROBLEMS AND RESEARCH DIRECTIONS

A common benchmark methodology. Section VI showed that reported recovery times are not comparable. The field would benefit from a small set of agreed scenarios, each fixing the detector, the failure type (carrier or silent), the controller distance and the measurement method, together with containerised reference implementations, so that a new scheme can be measured against the baselines on the same platform. Our artefact [20] is one step in this direction.

Detection at hardware speed and software cost. BFD at millisecond intervals is routine in hardware but fragile in software switches, and per-link sessions grow with the number of links. Traffic inference [40] needs no probes but is slower and depends on traffic. Data-plane probe generation in programmable switches is a promising middle ground [60], but published numbers rarely separate detection time from rerouting time.

Forwarding state. Protection's state cost grows with topology size (Table VI). Compression [31], aggregation [34], rule sharing [29] and header-carried paths [32], [46] reduce it, but their interaction with multi-failure guarantees and table update cost is not well understood.

The network after recovery. A fast-failover group chooses its bucket by liveness alone, so a backup can be selected even if it is already congested. Congestion-aware schemes [47], [48] plan for this, and hybrid re-optimisation repairs it later, but few evaluations measure post-recovery throughput and delay alongside outage time. Our own measurements, like most in the literature, use idle backup paths.

Multiple, correlated and control-plane failures. Most evaluations cut one link. Shared-risk link groups, node failures and the loss of the control channel itself [53], [55] are covered by few works, and the resilience that static forwarding tables can offer is itself an object of theoretical study [37].

Security of failure signalling. Any detector can be fooled. Forged BFD packets, suppressed probes or synthetic TCP

retransmissions could trigger unnecessary reroutes or hide real failures. Inference-based detection is exposed by design, because its input is generated by end hosts.

Learning-based recovery. Prediction accuracy and action quality [50], [51] do not show that recovery is faster. Evaluations should report inference latency and the end-to-end outage under the same conditions as non-learning baselines, and should state where in the recovery time line the model runs.

IX. Conclusion

SDN recovery research has moved from measuring the cost of a centralised controller, to making pre-installed protection affordable, to running detection and rerouting inside programmable data planes, and most recently to the quality of the network after recovery. Across this evolution, one conclusion has held: the controller round trip must be kept off the critical path to meet 50 ms, and once it is, failure detection is what remains. Our reproductions on a common testbed confirm both parts quantitatively. Restoration grows by one control-channel round trip per failure in two independently built testbeds, while protection is flat. BFD-driven protection reproduces the original recovery times within or just outside one standard deviation, and no LoS-driven scheme survives a silent failure. Published recovery times should therefore be read together with the detector, platform and controller distance that produced them. We hope that the taxonomy, the annotated benchmark table and the open artefact make such comparisons easier.

References

[1] M. Casado, M. J. Freedman, J. Pettit, J. Luo, N. McKeown, and S. Shenker, "Ethane: Taking control of the enterprise," *ACM SIGCOMM Computer Communication Review*, vol. 37, no. 4, pp. 1–12, 2007.

[2] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, and J. Turner, "OpenFlow: Enabling innovation in campus networks," *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 2, pp. 69–74, 2008.

[3] D. Kreutz, F. M. V. Ramos, P. Esteves Veríssimo, C. Esteve Rothenberg, S. Azodolmolky, and S. Uhlig, "Software-defined networking: A comprehensive survey," *Proceedings of the IEEE*, vol. 103, no. 1, pp. 14–76, 2015.

[4] S. Jain, A. Kumar, S. Mandal, J. Ong, L. Poutievski, A. Singh, S. Venkata, J. Wanderer, J. Zhou, M. Zhu, J. Zolla, U. Hölzle, S. Stuart, and A. Vahdat, "B4: Experience with a globally-deployed software defined WAN," in *Proc. ACM SIGCOMM*, 2013, pp. 3–14.

[5] A. Markopoulou, G. Iannaccone, S. Bhattacharyya, C.-N. Chuah, Y. Ganjali, and C. Diot, "Characterization of failures in an operational IP backbone network," *IEEE/ACM Transactions on Networking*, vol. 16, no. 4, pp. 749–762, 2008.

[6] P. Gill, N. Jain, and N. Nagappan, "Understanding network failures in data centers: Measurement, analysis, and implications," in *Proc. ACM SIGCOMM*, 2011, pp. 350–361.

[7] B. Heller, R. Sherwood, and N. McKeown, "The controller placement problem," in *Proc. 1st Workshop on Hot Topics in Software Defined Networks (HotSDN)*, 2012, pp. 7–12.

[8] ITU-T, "Types and characteristics of SDH network protection architectures," ITU-T Recommendation G.841, Oct. 1998.

[9] D. Staessens, S. Sharma, D. Colle, M. Pickavet, and P. Demeester, "Software defined networking: Meeting carrier grade requirements," in *Proc. 18th IEEE Workshop on Local and Metropolitan Area Networks (LANMAN)*, 2011, pp. 1–6.

[10] S. Sharma, D. Staessens, D. Colle, M. Pickavet, and P. Demeester, "OpenFlow: Meeting carrier-grade recovery requirements," *Computer Communications*, vol. 36, no. 6, pp. 656–665, 2013.

[11] V. Sharma and F. Hellstrand, "Framework for multi-protocol label switching (MPLS)-based recovery," RFC 3469, IETF, Feb. 2003.

[12] P. C. da Rocha Fonseca and E. S. Mota, "A survey on fault management in software-defined networks," *IEEE Communications Surveys & Tutorials*, vol. 19, no. 4, pp. 2284–2321, 2017.

[13] J. Chen, J. Chen, F. Xu, M. Yin, and W. Zhang, "When software defined networks meet fault tolerance: A survey," in *Algorithms and Architectures for Parallel Processing (ICA3PP 2015)*, ser. Lecture Notes in Computer Science. Springer, 2015, pp. 351–368.

[14] J. Ali, G.-m. Lee, B.-h. Roh, D. K. Ryu, and G. Park, "Software-defined networking approaches for link failure recovery: A survey," *Sustainability*, vol. 12, no. 10, p. 4255, 2020.

[15] S. Alhiyari, N. N. Daud, and S. H. Ab Hamid, "A survey of link failure detection and recovery in software-defined networks," *Computers, Materials & Continua*, vol. 82, no. 1, pp. 103–137, 2025.

[16] N. Khan, R. B. Salleh, A. Koubaa, Z. Khan, M. K. Khan, and I. Ali, "Data plane failure and its recovery techniques in SDN: A systematic literature review," *Journal of King Saud University – Computer and Information Sciences*, vol. 35, no. 3, pp. 176–201, 2023.

[17] M. Chiesa, A. Kamisiński, J. Rak, G. Rétvári, and S. Schmid, "A survey of fast-recovery mechanisms in packet-switched networks," *IEEE Communications Surveys & Tutorials*, vol. 23, no. 2, pp. 1253–1301, 2021.

[18] A. Mazloum, E. Kfoury, J. Gomez, and J. Crichigno, "A survey on rerouting techniques with P4 programmable data plane switches," *Computer Networks*, vol. 230, p. 109795, 2023.

[19] N. L. M. van Adrichem, B. J. van Asten, and F. A. Kuipers, "Fast recovery in software-defined networks," in *Proc. 3rd European Workshop on Software Defined Networks (EWSDN)*, 2014, pp. 61–66.

[20] "sdn-fast-recovery: Replication of van Adrichem et al. (EWSDN 2014) on Mininet and Open vSwitch," https://github.com/baalajivinay/sdn-fast-recovery, container image `ghcr.io/baalajivinay/sdn-fast-recovery:v1.0`, 2026.

[21] Open Networking Foundation, "OpenFlow switch specification, version 1.3.0 (wire protocol 0x04)," Open Networking Foundation, Tech. Rep. ONF TS-006, Jun. 2012.

[22] P. Bosshart, D. Daly, G. Gibb, M. Izzard, N. McKeown, J. Rexford, C. Schlesinger, D. Talayco, A. Vahdat, G. Varghese, and D. Walker, "P4: Programming protocol-independent packet processors," *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 3, pp. 87–95, 2014.

[23] B. Pfaff, J. Pettit, T. Koponen, E. J. Jackson, A. Zhou, J. Rajahalme, J. Gross, A. Wang, J. Stringer, P. Shelar, K. Amidon, and M. Casado, "The design and implementation of Open vSwitch," in *Proc. 12th USENIX Symp. Networked Systems Design and Implementation (NSDI)*, 2015, pp. 117–130.

[24] B. Lantz, B. Heller, and N. McKeown, "A network in a laptop: Rapid prototyping for software-defined networks," in *Proc. 9th ACM SIGCOMM Workshop on Hot Topics in Networks (HotNets)*, 2010, pp. 1–6.

[25] Open Networking Foundation, "OpenFlow switch specification, version 1.1.0 (wire protocol 0x02)," Open Networking Foundation, Tech. Rep. ONF TS-002, Feb. 2011.

[26] M. Desai and T. Nandagopal, "Coping with link failures in centralized control plane architectures," in *Proc. 2nd Int. Conf. Communication Systems and Networks (COMSNETS)*, 2010, pp. 1–10.

[27] S. Sharma, D. Staessens, D. Colle, M. Pickavet, and P. Demeester, "Enabling fast failure recovery in OpenFlow networks," in *Proc. 8th Int. Workshop on the Design of Reliable Communication Networks (DRCN)*, 2011, pp. 164–171.

[28] J. Kempf, E. Bellagamba, A. Kern, D. Jocha, A. Takács, and P. Sköldström, "Scalable fault management for OpenFlow," in *Proc. IEEE Int. Conf. Communications (ICC)*, 2012, pp. 6606–6610.

[29] P. M. Mohan, T. Truong-Huu, and M. Gurusamy, "TCAM-aware local rerouting for fast and efficient failure recovery in software defined networks," in *Proc. IEEE Global Communications Conf. (GLOBECOM)*, 2015, pp. 1–6.

[30] D. Merling, W. Braun, and M. Menth, "Efficient data plane protection for SDN," in *Proc. 4th IEEE Conf. Network Softwarization and Workshops (NetSoft)*, 2018, pp. 10–18.

[31] B. Stephens, A. L. Cox, and S. Rixner, "Scalable multi-failure fast failover via forwarding table compression," in *Proc. ACM Symp. SDN Research (SOSR)*, 2016, pp. 1–12.

[32] R. M. Ramos, M. Martinello, and C. Esteve Rothenberg, "SlickFlow: Resilient source routing in data center networks unlocked by OpenFlow," in *Proc. 38th IEEE Conf. Local Computer Networks (LCN)*, 2013, pp. 606–613.

[33] V. Liu, D. Halperin, A. Krishnamurthy, and T. Anderson, "F10: A fault-tolerant engineered network," in *Proc. 10th USENIX Symp. Networked Systems Design and Implementation (NSDI)*, 2013, pp. 399–412.

[34] X. Zhang, Z. Cheng, R. Lin, L. He, S. Yu, and H. Luo, "Local fast reroute with flow aggregation in software defined networks," *IEEE Communications Letters*, vol. 21, no. 4, pp. 785–788, 2017.

[35] B. Stephens, A. L. Cox, and S. Rixner, "Plinko: Building provably resilient forwarding tables," in *Proc. 12th ACM Workshop on Hot Topics in Networks (HotNets)*, 2013, pp. 1–7.

[36] M. Borokhovich, L. Schiff, and S. Schmid, "Provable data plane connectivity with local fast failover: Introducing OpenFlow graph algorithms," in *Proc. 3rd Workshop on Hot Topics in Software Defined Networking (HotSDN)*, 2014, pp. 121–126.

[37] M. Chiesa, I. Nikolaevskiy, S. Mitrović, A. Panda, A. Gurtov, A. Mądry, M. Schapira, and S. Shenker, "The quest for resilient (static) forwarding tables," in *Proc. IEEE INFOCOM*, 2016, pp. 1–9.

[38] G. Bianchi, M. Bonola, A. Capone, and C. Cascone, "OpenState: Programming platform-independent stateful OpenFlow applications inside the switch," *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 2, pp. 44–51, 2014.

[39] C. Cascone, L. Pollini, D. Sanvito, A. Capone, and B. Sansò, "SPIDER: Fault resilient SDN pipeline with recovery delay guarantees," in *Proc. IEEE NetSoft Conf. and Workshops*, 2016, pp. 296–302.

[40] T. Holterbach, E. Costa Molero, M. Apostolaki, A. Dainotti, S. Vissicchio, and L. Vanbever, "Blink: Fast connectivity recovery entirely in the data plane," in *Proc. 16th USENIX Symp. Networked Systems Design and Implementation (NSDI)*, 2019, pp. 161–176.

[41] R. Sedar, M. Borokhovich, M. Chiesa, G. Antichi, and S. Schmid, "Supporting emerging applications with low-latency failover in P4," in *Proc. Workshop on Networking for Emerging Applications and Technologies (NEAT)*, 2018, pp. 52–57.

[42] M. Chiesa, R. Sedar, G. Antichi, M. Borokhovich, A. Kamisiński, G. Nikolaidis, and S. Schmid, "PURR: A primitive for reconfigurable fast reroute," in *Proc. 15th ACM Int. Conf. Emerging Networking Experiments and Technologies (CoNEXT)*, 2019, pp. 1–14.

[43] T. Qu, R. Joshi, M. C. Chan, B. Leong, D. Guo, and Z. Liu, "SQR: In-network packet loss recovery from link failures for highly reliable datacenter networks," in *Proc. 27th IEEE Int. Conf. Network Protocols (ICNP)*, 2019, pp. 1–12.

[44] S. Lindner, D. Merling, M. Häberle, and M. Menth, "P4-Protect: 1+1 path protection for P4," in *Proc. 3rd P4 Workshop in Europe (EuroP4)*, 2020, pp. 21–27.

[45] K. Subramanian, A. Abhashkumar, L. D'Antoni, and A. Akella, "D2R: Policy-compliant fast reroute," in *Proc. ACM SIGCOMM Symp. SDN Research (SOSR)*, 2021, pp. 148–161.

[46] Z. Li, Y. Hu, J. Wu, and J. Lu, "P4Resilience: Scalable resilience for multi-failure recovery in SDN with programmable data plane," *Computer Networks*, vol. 208, p. 108896, 2022.

[47] Z. Cheng, X. Zhang, Y. Li, S. Yu, R. Lin, and L. He, "Congestion-aware local reroute for fast failure recovery in software-defined networks," *Journal of Optical Communications and Networking*, vol. 9, no. 11, pp. 934–944, 2017.

[48] Z. Zhu, Q. Li, S. Xia, and M. Xu, "CAFFE: Congestion-aware fast failure recovery in software defined networks," in *Proc. 27th Int. Conf. Computer Communication and Networks (ICCCN)*, 2018, pp. 1–9.

[49] D. De Lima, F. Vogt, A. T. da Silva, F. Rodriguez Cesen, and C. Esteve Rothenberg, "RESISTING: A new fast-reroute mechanism with packet distribution on P4-programmable switches," in *Proc. IEEE Conf. Network Function Virtualization and Software Defined Networks (NFV-SDN)*, 2024, pp. 1–6.

[50] D. Bishla and B. Kumar, "Performance enhancement in hybrid SDN using advanced deep learning with multi-objective optimization frameworks under heterogeneous environments," *International Journal of Communication Systems*, vol. 38, no. 3, p. e5989, 2025.

[51] E. H. Makiyah, M. N. Rasool, and F. A. Rawdhan, "A reinforcement learning framework for self-healing fault recovery in intent-based SDNs," *Discover Networks*, vol. 2, no. 1, p. 14, 2026.

[52] H. Kim, M. Schlansker, J. R. Santos, J. Tourrilhes, Y. Turner, and N. Feamster, "CORONET: Fault tolerance for software defined networks," in *Proc. 20th IEEE Int. Conf. Network Protocols (ICNP)*, 2012, pp. 1–2.

[53] S. Sharma, D. Staessens, D. Colle, M. Pickavet, and P. Demeester, "Fast failure recovery for in-band OpenFlow networks," in *Proc. 9th Int. Conf. Design of Reliable Communication Networks (DRCN)*, 2013, pp. 52–59.

[54] A. Sgambelluri, A. Giorgetti, F. Cugini, F. Paolucci, and P. Castoldi, "OpenFlow-based segment protection in Ethernet networks," *Journal of Optical Communications and Networking*, vol. 5, no. 9, pp. 1066–1075, 2013.

[55] D. Kotani and Y. Okabe, "Fast failure detection of OpenFlow channels," in *Proc. Asian Internet Engineering Conf. (AINTEC)*, 2015, pp. 32–39.

[56] N. Katta, M. Hira, C. Kim, A. Sivaraman, and J. Rexford, "HULA: Scalable load balancing using programmable data planes," in *Proc. ACM Symp. SDN Research (SOSR)*, 2016, pp. 1–12.

[57] B. J. van Asten, N. L. M. van Adrichem, and F. A. Kuipers, "Scalability and resilience of software-defined networking: An overview," arXiv:1408.6760, 2014.

[58] D. Katz and D. Ward, "Bidirectional forwarding detection (BFD)," RFC 5880, IETF, Jun. 2010.

[59] S. S. W. Lee, K.-Y. Li, K.-Y. Chan, G.-H. Lai, and Y.-C. Chung, "Path layout planning and software based fast failure detection in survivable OpenFlow networks," in *Proc. 10th Int. Conf. Design of Reliable Communication Networks (DRCN)*, 2014, pp. 1–8.

[60] C. Cascone, D. Sanvito, L. Pollini, A. Capone, and B. Sansò, "Fast failure detection and recovery in SDN with stateful data plane," *International Journal of Network Management*, vol. 27, no. 2, p. e1957, 2017.

[61] A. Atlas and A. Zinin, "Basic specification for IP fast reroute: Loop-free alternates," RFC 5286, IETF, Sep. 2008.

[62] S. Bryant, C. Filsfils, S. Previdi, M. Shand, and N. So, "Remote loop-free alternate (LFA) fast reroute (FRR)," RFC 7490, IETF, Apr. 2015.

[63] M. Shand and S. Bryant, "IP fast reroute framework," RFC 5714, IETF, Jan. 2010.